the window, passing it the name of the window and the image variable. After that, *WaitKey* is used so that you can see the image for a sizeable period of time (similar to a delay function). Finally, *DestroyWindow* destroys the window and frees the resources it used.

Converting an RGB image to grey-scale format

Most digital images seen in everyday life use the RGB colour model, in which every pixel has three components—red, green and blue. By varying the amount of each component, a pixel gets a unique colour. Grey-scale is another popular colour model, with only one component, which varies from 0-255; 0 represents black, and 255 white (for an 8-bit image). The code below, *Rgb2gray.py*, converts an image from RGB to grey-scale:

```
#!/usr/bin/python
import cv
src = cv.LoadImage("/home/jayneil/12.png")
cv.NamedWindow("RGB", cv.CV_WINDOW_AUTOSIZE)
cv.ShowImage("RGB", src)
cv.WaitKey(500)
cv.DestroyWindow("RGB")
dst = cv.CreateImage(cv.GetSize(src), cv.IPL_DEPTH_8U, 1)
cv.CvtColor(src, dst, cv.CV_RGB2GRAY)
cv.NamedWindow("GRAY", cv.CV_WINDOW_AUTOSIZE)
cv.ShowImage("GRAY", dst)
cv.WaitKey(1000)
cv.DestroyWindow("GRAY")
```

Running this should result in output similar to Figure 2. I'll skip the functions I've already explained. In the eighth line, we have created our own image, similar to the source image *(src)*. *GetSize* returns the properties of the source image—so our new image has the same dimensions as the source. *CreateImage* takes three parameters: the image dimensions, the colour depth of the image (8-bit or 16-bit) and the number of channels (which, for grey-scale, is one). We used the *CvtColor* function to convert the image to grey-scale. This function has three parameters. The first one is the name of the image which we want to convert and the second denotes what type of conversion we want to do. The third parameter performs the type of conversion desired. In this case it is from RGB to grey-scale.

Drawing a circle on an image

You can also edit or manipulate a given image, for example, like drawing a circle on it. The following code (*Circle.py*) does this:

```
#!/usr/bin/python
import cv
src = cv.LoadImage("/home/jayneil/12.png")
```

Figure 1: Read and display an image

Figure 2: A grey-scale image

Figure 3: Drawing a circle on a given image

Figure 4: The Laplacian of an image

```
print(src)
cv.NamedWindow("RGB", cv.CV_WINDOW_AUTOSIZE)
cv.ShowImage("RGB", src)
cv.WaitKey(1000)
cv.DestroyWindow("RGB")
thickness = 3
lineType=8
Point = (292,300)
cv.Circle(src, Point,100, cv.Scalar(0,0,255), thickness, lineType)
cv.NamedWindow("Circle_on_image", cv.CV_WINDOW_AUTOSIZE)
cv.ShowImage("Circle_on_image", src)
```